

Assignment

Introduction to PostgreSQL, pgAdmin, Schema & Datatypes

Section A — Conceptual Questions

Q1. What is PostgreSQL? List any four key features that differentiate it from other RDBMS.

Answer: PostgreSQL is an open-source, object-relational database management system (ORDBMS) used to store, manage, and retrieve structured data. It is known for being reliable, powerful, and highly customizable.

Four key features are:

- **Open-source and free:** PostgreSQL can be used and modified without licensing costs.
- **ACID compliant:** It supports reliable transactions with Atomicity, Consistency, Isolation, and Durability.
- **Advanced data types:** It supports JSON/JSONB, arrays, UUIDs, geometric data, and custom data types.
- **Extensibility:** Developers can create custom functions, operators, data types, extensions, and indexes.

Q2. What are the five broad categories of SQL commands? Name one command under each category.

Answer: SQL commands are generally divided into five categories:

1. **Data Definition Language (DDL):** Defines and modifies the structure of the database. Example- CREATE
2. **Data Manipulation Language (DML):** Inserts, updates, and deletes data in the database. Example- INSERT

3. **Data Query Language (DQL):** Retrieves data from the database.

Example- SELECT

4. **Data Control Language Controls (DCL):** user access and permissions.

Example- GRANT

5. **Transaction Control Language (TCL):** Manages database transactions.

Example- COMMIT

Q3. What is pgAdmin used for? Briefly describe the purpose of the Query Tool and the Dashboard.

Answer: pgAdmin is a graphical administration and management tool for PostgreSQL. It allows developers and database administrators to manage databases without having to perform everything through the command line.

- **Query Tool:** It is used to write and execute SQL queries. We can run commands such as SELECT, INSERT, UPDATE, DELETE, CREATE TABLE, etc., and view the results.
- **Dashboard:** It provides an overview of the PostgreSQL server, including information about server activity, sessions, transactions, and performance-related statistics.

Q4. What is a Schema in PostgreSQL? Why is the default schema called 'public'?

Answer: A schema in PostgreSQL is like a logical container or namespace inside a database. It is used to organize database objects such as tables, views, functions, and sequences.

For example, one database can have schemas such as:

company

├─ hr

├─ sales

└─ finance

The public schema is the default schema created in a new PostgreSQL database. It is called public because it is traditionally available for general use by users who have the appropriate permissions.

For example:

```
CREATE TABLE employees (  
    id INT,  
    name TEXT  
);
```

If no schema is specified, the table is normally created in the public schema, assuming the user's search_path and permissions allow it.

Q5. Differentiate between CHAR(n), VARCHAR(n) and TEXT data types.

Answer: The main difference is how they handle string length.

- CHAR(n): Fixed-length string. If the value is shorter than n, PostgreSQL pads it with spaces.
- VARCHAR(n): Variable-length string with a maximum length of n.
- TEXT: Variable-length string with no explicitly specified maximum length.

For example:

name CHAR (10) → stores a fixed-length character value.

name VARCHAR (100) → allows a string up to 100 characters.

description TEXT → allows text without specifying a maximum length.

Q6. What is the difference between TIMESTAMP and TIMESTAMPTZ?

Answer: The main difference is **time zone handling**.

- TIMESTAMP stores a date and time **without time-zone information**.
- TIMESTAMPTZ means **timestamp with time zone**. PostgreSQL converts the supplied value to UTC internally and displays it according to the current session's time zone.

For example:

```
CREATE TABLE events (  
    event_time TIMESTAMP,  
    created_at TIMESTAMPTZ  
);
```

If the application needs to record an event that represents a specific moment in time—such as a payment, login, or order creation—TIMESTAMPTZ is generally more appropriate.

Q7. What is the purpose of the SERIAL data type? What does it internally create?

Answer: SERIAL is a PostgreSQL convenience feature used to automatically generate sequential integer values, commonly for **primary key IDs**.

For example:

```
CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name TEXT  
);
```

When a new employee is inserted without specifying id, PostgreSQL automatically generates the next value.

Internally, SERIAL is essentially shorthand for creating:

- An **integer column**
- A **sequence**
- A **default value** that gets the next value from that sequence

Conceptually, this: id SERIAL

is similar to: id INTEGER NOT NULL DEFAULT nextval('some_sequence')

along with a sequence object.

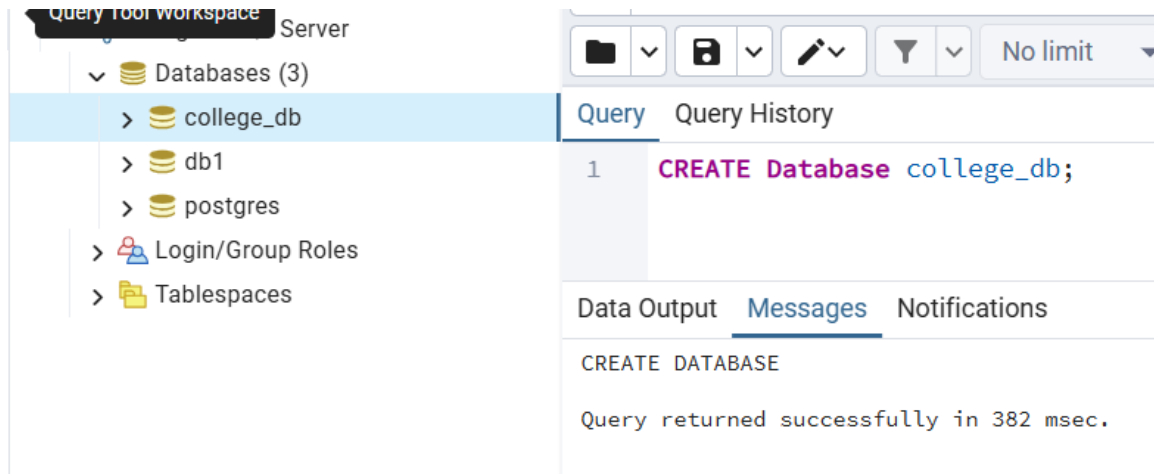
Section B — Hands-on pgAdmin Practice

Server, Database, Schemas and Data Types

Task 1: Server & Database Setup

SQL used:

```
CREATE DATABASE college_db;
```

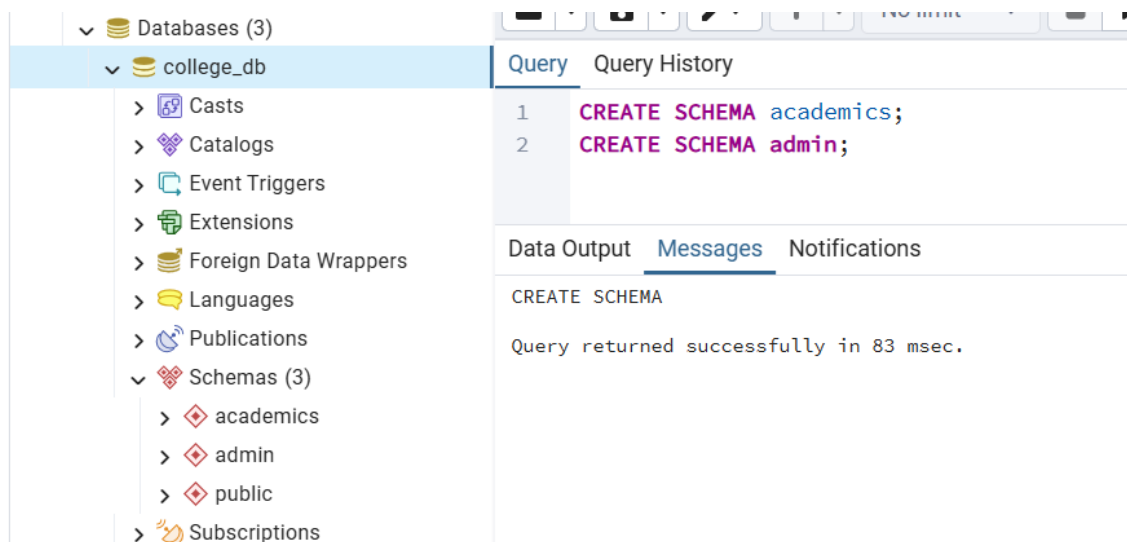


Task 2: Schema Creation

SQL Used: Inside college_db, create a schemas named academics and admin

```
CREATE SCHEMA academics;
```

```
CREATE SCHEMA admin;
```



Query to List All Schemas:

```
SELECT schema_name
FROM information_schema.schemata
ORDER BY schema_name;
```

The screenshot shows a database management interface. On the left, a tree view displays the database structure, with 'college_db' selected. The 'Schemas (3)' folder is expanded, showing 'academics', 'admin', and 'public'. The main panel is divided into 'Query' and 'Query History' tabs. The 'Query' tab contains the SQL query:

```
-- CREATE SCHEMA academics;
-- CREATE SCHEMA admin;

SELECT schema_name
FROM information_schema.schemata
ORDER BY schema_name;
```

 Below the query editor are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with the results of the query. The table has two columns: 'schema_name' and 'name'. The results are as follows:

	schema_name	name
1	academics	
2	admin	
3	information_sche...	
4	pg_catalog	
5	pg_toast	
6	public	

Task 3: Explore Data Types by Creating Tables

Create TABLE statement:

```
CREATE TABLE academics.students (  
    student_id SERIAL PRIMARY KEY,  
    full_name VARCHAR(100) NOT NULL,  
    dob DATE,  
    email TEXT,  
    cgpa NUMERIC(3,2),  
    is_active BOOLEAN DEFAULT TRUE,  
    skills TEXT[],  
    profile JSONB,  
    enrolled_at TIMESTAMPTZ DEFAULT NOW()  
);
```

The screenshot displays a database management tool interface. On the left, a tree view shows the database structure, with 'Tables (1)' expanded to show the 'students' table. The 'students' table is selected, and its properties (Columns, Constraints, Indexes, RLS Policies, Rules, Triggers) are visible. The main area shows the SQL statement being executed, with line numbers 4 through 19. The statement is the same CREATE TABLE statement shown above. Below the SQL editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the message 'CREATE TABLE' and 'Query returned successfully in 91 msec.'

```
4  -- SELECT schema_name  
5  -- FROM information_schema.schemata  
6  -- ORDER BY schema_name;  
7  
8  CREATE TABLE academics.students (  
9      student_id SERIAL PRIMARY KEY,  
10     full_name VARCHAR(100) NOT NULL,  
11     dob DATE,  
12     email TEXT,  
13     cgpa NUMERIC(3,2),  
14     is_active BOOLEAN DEFAULT TRUE,  
15     skills TEXT[],  
16     profile JSONB,  
17     enrolled_at TIMESTAMPTZ DEFAULT NOW()  
18 );  
19
```

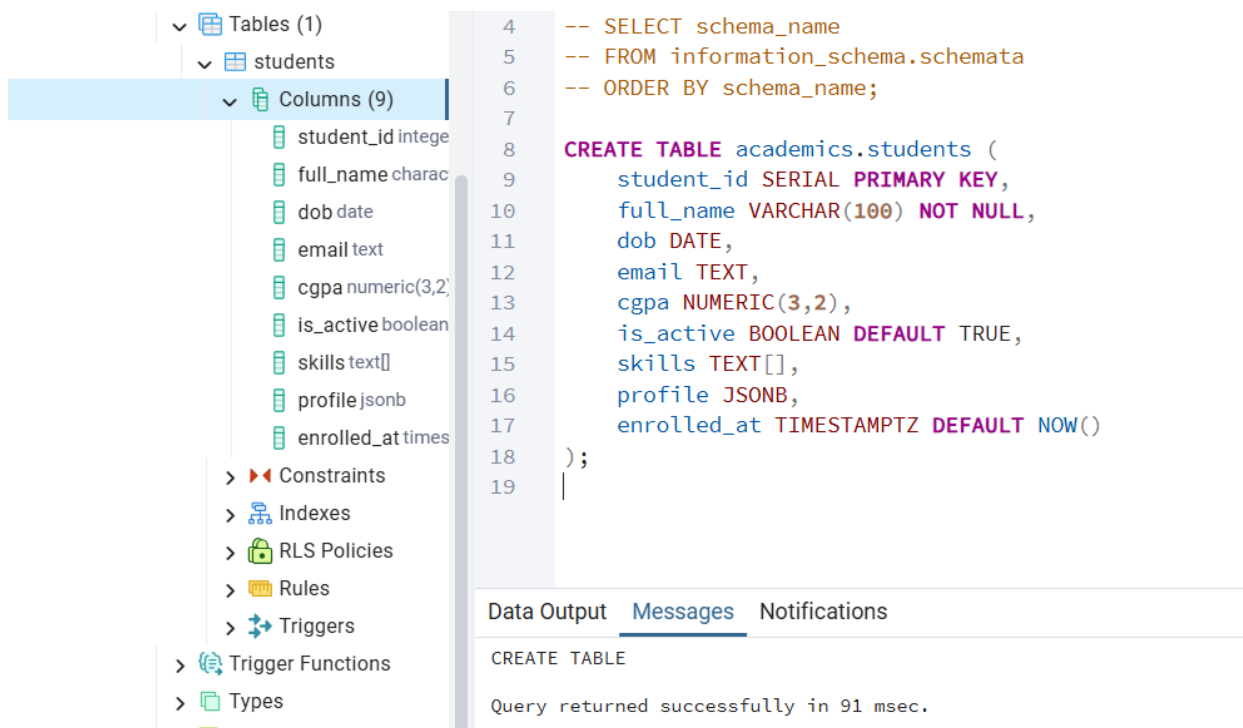
Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 91 msec.

Task 4: Verify Work

- Expand college_db -> Schemas -> academics -> Tables and confirm students is present.
- Right-click students -> Properties -> Columns and verify that all nine columns match Task 3.



The screenshot displays a database management interface. On the left, a tree view shows the database structure: 'Tables (1)' is expanded, showing 'students', which is further expanded to show 'Columns (9)'. The columns listed are: student_id integer, full_name character, dob date, email text, cgpa numeric(3,2), is_active boolean, skills text[], profile jsonb, and enrolled_at timestamp. Below the columns, other properties like Constraints, Indexes, RLS Policies, Rules, Triggers, Trigger Functions, and Types are visible. On the right, a SQL editor shows the following code:

```
4 -- SELECT schema_name
5 -- FROM information_schema.schemata
6 -- ORDER BY schema_name;
7
8 CREATE TABLE academics.students (
9     student_id SERIAL PRIMARY KEY,
10    full_name VARCHAR(100) NOT NULL,
11    dob DATE,
12    email TEXT,
13    cgpa NUMERIC(3,2),
14    is_active BOOLEAN DEFAULT TRUE,
15    skills TEXT[],
16    profile JSONB,
17    enrolled_at TIMESTAMPTZ DEFAULT NOW()
18 );
19
```

At the bottom, a tabbed interface shows 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, displaying the message: 'CREATE TABLE' and 'Query returned successfully in 91 msec.'

Query to List Column Names and Data Types:

```
SELECT

    column_name,

    data_type

FROM information_schema.columns

WHERE table_schema = 'academics'

    AND table_name = 'students'

ORDER BY ordinal_position;
```

The screenshot shows a database management interface. On the left, a tree view displays the database structure: > 1.3 Sequences, > Tables (1), > students, > Columns (9). The 'Columns (9)' folder is expanded, showing a list of columns: student_id integer, full_name character varying, dob date, email text, cgpa numeric(3,2), is_active boolean, skills text[], profile jsonb, and enrolled_at timestamp with time zone. Below this, other database objects like Constraints, Indexes, RLS Policies, Rules, Triggers, Trigger Functions, Types, Views, and admin are listed. On the right, a SQL editor shows the query: SELECT column_name, data_type FROM information_schema.columns WHERE table_schema = 'academics' AND table_name = 'students' ORDER BY ordinal_position;. Below the editor, the 'Data Output' tab is active, displaying a table with 9 rows and 2 columns: column_name and data_type. The table contains the following data:

	column_name	data_type
1	student_id	integer
2	full_name	character varying
3	dob	date
4	email	text
5	cgpa	numeric
6	is_active	boolean
7	skills	ARRAY
8	profile	jsonb
9	enrolled_at	timestamp with time zo...

Complete SQL Used in above Assignment

-- Task 1: Run while connected to the postgres maintenance database

```
CREATE DATABASE college_db;
```

-- After creating the database, connect/open Query Tool for college_db

-- Task 2

```
CREATE SCHEMA academics;
```

```
CREATE SCHEMA admin;
```

SELECT schema_name

```
FROM information_schema.schemata
```

```
ORDER BY schema_name;
```

-- Task 3

```
CREATE TABLE academics.students (  
    student_id SERIAL PRIMARY KEY,  
    full_name VARCHAR(100) NOT NULL,  
    dob DATE,  
    email TEXT,  
    cgpa NUMERIC(3,2),  
    is_active BOOLEAN DEFAULT TRUE,  
    skills TEXT[],  
    profile JSONB,  
    enrolled_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Task 4

```
SELECT  
    column_name,  
    data_type  
FROM information_schema.columns  
WHERE table_schema = 'academics'  
    AND table_name = 'students'  
ORDER BY ordinal_position;
```

Section C — Challenge Exercise

Library Database – PostgreSQL/pgAdmin

Task 1: Create the Database

SQL used:

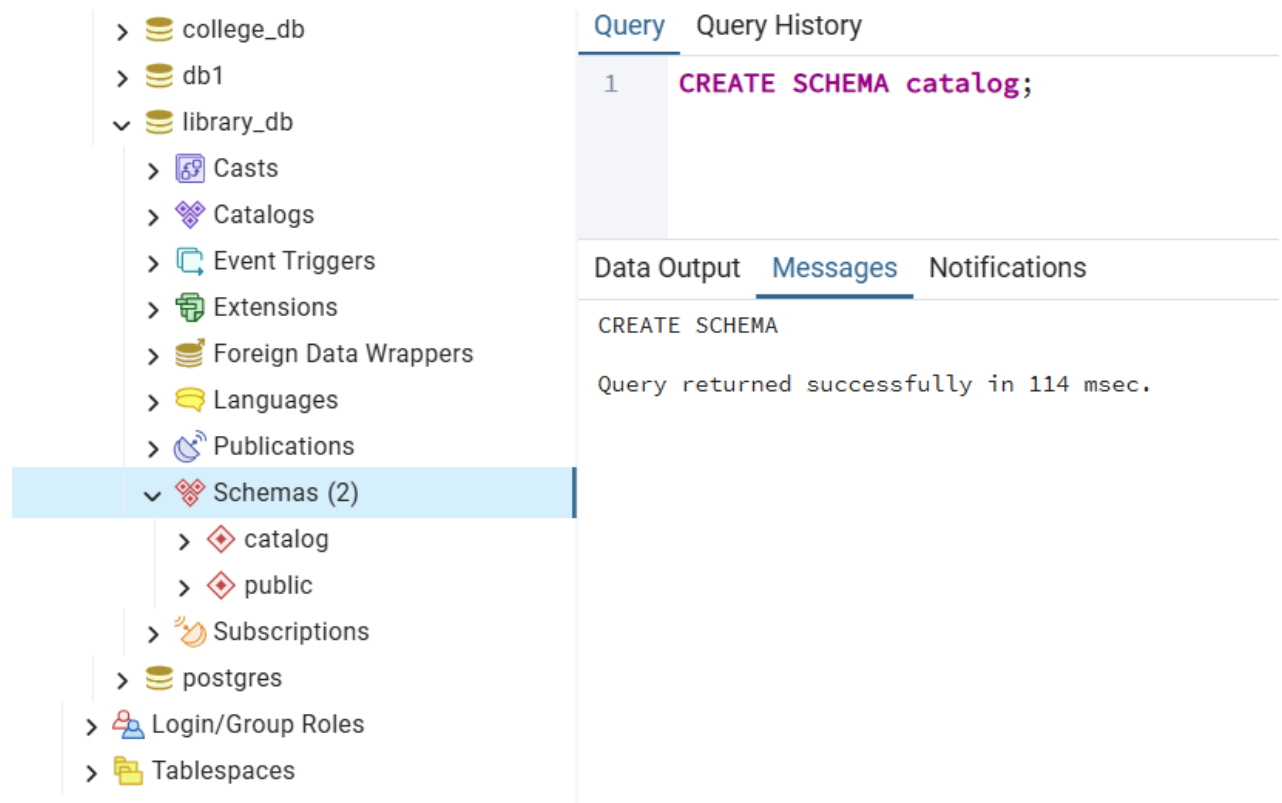
```
CREATE DATABASE library_db;
```

The screenshot displays the pgAdmin web interface. On the left, the 'Postgres SQL Server' tree is expanded, showing a list of databases: college_db, db1, library_db (highlighted in blue), and postgres. Below the databases are 'Login/Group Roles' and 'Tablespaces'. On the right, the 'Query' tab is active, showing the SQL command 'CREATE DATABASE library_db;' in a syntax-highlighted editor. Below the editor, the 'Messages' tab is selected, displaying the message 'CREATE DATABASE' and 'Query returned successfully in 313 msec.'

Task 2: Create the Schema

SQL used: Inside library_db, create a schema named catalog.

```
CREATE SCHEMA catalog;
```



The screenshot displays a database management interface. On the left, a tree view shows the database structure. Under the 'library_db' database, the 'Schemas (2)' folder is expanded, revealing two schemas: 'catalog' and 'public'. The 'catalog' schema is highlighted. On the right, the 'Messages' tab is active, showing the execution of the 'CREATE SCHEMA catalog;' query. The message indicates that the query was returned successfully in 114 msec.

Task 3: Create the Book Table

SQL used:

```
CREATE TABLE catalog.books (  
  book_id SERIAL PRIMARY KEY,  
  title VARCHAR(200),  
  author VARCHAR(150),  
  isbn VARCHAR(20),  
  price NUMERIC(10,2),  
  published_date DATE,  
  is_available BOOLEAN DEFAULT TRUE,  
  extra_info JSONB  
)
```

The screenshot shows a database management tool interface. On the left, a tree view displays the database structure: Languages, Publications, Schemas (2), and a sub-tree for the 'catalog' schema. The 'catalog' schema contains various objects like Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, Tables (1), Trigger Functions, and Types. The 'Tables (1)' folder is expanded, showing the 'books' table. On the right, a SQL editor displays the following query:

```

1  -- CREATE SCHEMA catalog;
2
3  CREATE TABLE catalog.books (
4    book_id SERIAL PRIMARY KEY,
5    title VARCHAR(200),
6    author VARCHAR(150),
7    isbn VARCHAR(20),
8    price NUMERIC(10,2),
9    published_date DATE,
10   is_available BOOLEAN DEFAULT TRUE,
11   extra_info JSONB
12 )

```

Below the SQL editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the following output:

```

CREATE TABLE
Query returned successfully in 62 msec.

```

Task 4: Verify Work

- Expand library_db -> Schemas -> catalog -> Tables and confirm books is present.
- Right-click books -> Properties -> Columns and verify that all eight columns match the table specification.

The screenshot shows the same database management tool interface. The 'catalog' schema is expanded, and the 'books' table is selected. The 'Columns' tab is active, showing the following columns:

Column Name	Data Type	Primary Key	Nullable	Default Value
book_id	SERIAL	Yes	No	
title	VARCHAR(200)	No	Yes	
author	VARCHAR(150)	No	Yes	
isbn	VARCHAR(20)	No	Yes	
price	NUMERIC(10,2)	No	Yes	
published_date	DATE	No	Yes	
is_available	BOOLEAN	No	No	TRUE
extra_info	JSONB	No	Yes	

On the right, a SQL editor displays the following query:

```

4
5
6  CREATE TABLE catalog.books (
7    book_id SERIAL PRIMARY KEY,
8    title VARCHAR(200),
9    author VARCHAR(150),
10   isbn VARCHAR(20),
11   price NUMERIC(10,2),
12   published_date DATE,
13   is_available BOOLEAN DEFAULT TRUE,
14   extra_info JSONB
15 )

```

Below the SQL editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the following output:

```

CREATE TABLE
Query returned successfully in 62 msec.

```

Complete SQL Used in above Assignment

-- Task 1: Run while connected to the postgres maintenance database

```
CREATE DATABASE library_db;
```

-- After creating the database, connect/open Query Tool for library_db

-- Task 2

```
CREATE SCHEMA catalog;
```

-- Task 3

```
CREATE TABLE catalog.books (  
    book_id SERIAL PRIMARY KEY,  
    title VARCHAR(200),  
    author VARCHAR(150),  
    isbn VARCHAR(20),  
    price NUMERIC(10,2),  
    published_date DATE,  
    is_available BOOLEAN DEFAULT TRUE,  
    extra_info JSONB  
);
```

-- Task 4

```
SELECT  
    column_name,  
    data_type  
FROM information_schema.columns  
WHERE table_schema = 'catalog'  
    AND table_name = 'books'  
ORDER BY ordinal_position;
```